Assemblers: A Modular, Reconfigurable Manipulator for Autonomous in-Space Assembly

John R. Cooper,* James H. Neilan,† Matthew K. Mahlin,‡ and Laura M. White,§
NASA Langley Research Center, Hampton, VA, 23681

This paper presents a modular, reconfigurable robotic manipulator for autonomous in-space assembly. On-orbit and planetary surface assembly is a challenging domain that encompasses various technological thrusts to support human exploration, science, and technology demonstration missions. The system architecture presented here addresses the need for robotic assembly of structures in locations that prohibit constant human oversight and/or teleoperation. This system is capable of scheduling, reconfiguring itself, and executing structural assembly tasks with the ability to assess construction and correct errors during the assembly as needed. An example concept of operations is presented. Details of the mechanical and electrical design of the robotic modules as well as the dynamic modeling approach, control algorithms, and sensing and perception systems are also provided.

I. Introduction

ASSEMBLERS is a modular, reconfigurable robotic system designed to perform autonomous assembly and construction on the lunar surface. The system is currently under development at NASA Langley Research Center (LaRC). This paper serves as an overview of the project and presents preliminary designs for hardware, algorithms, and software that will be employed.

An Assemblers manipulator is composed of any number of stacked modules. Each module is a Stewart platform (SP) which is a fully functional six degree-of-freedom (DOF) robot on its own. Each SP consists of two plates connected by six linear actuators. The SP mechanism transforms the DOF of each of the six actuators to a 6 DOF pose of the top plate relative to the bottom plate. The SP modules employed by Assemblers also offer high redundancy and structural stiffness [1]. Additionally, the end-effector of an Assemblers manipulator is interchangeable and can be swapped out depending on the task.

A stacked SP architecture was pioneered by LaRC in the 1980s [2]. Further research was conducted by Miura and Furuya with the adaptive structure concept [3] and Hamlin, et al. with TETROBOT [1]. While stacked SPs based on the Intelligent Precision Jigging Robot paradigm were involved in more recent LaRC developments [4], the majority of these efforts were performed around thirty years ago. Major advances over the past decade have led to sensors and computers that are much lighter weight and less expensive than those during the 1980s. Furthermore, advances in state estimation, modern control theory, and most recently machine learning have converged such that the vision described herein can be made practical for the first time.

A practical assembly system will be able to autonomously and robustly manipulate, assemble, and service objects with limited supervision. Assemblers is designed to operate as a fully autonomous system. The modules will self-assemble into appropriately configured manipulators to build a structure with a single start command from a human operator. However, the ability for a human operator to override actions will be maintained at all levels of operation. This architecture constitutes a robust assembly system that can perform its nominal mission with minimal human interaction and still allow an operator to manually handle scenarios that lie far outside expected uncertainty levels.

On-orbit servicing, assembly, and manufacturing (OSAM) is set to revolutionize the space ecosystem. Many proposed missions rely on large structures such as habitats, power trusses, science equipment, telescopes, refueling stations, and spacecraft. The current practice for missions involving large space structures is to develop complex deployment mechanisms to allow for packaging within a limited launch vehicle volume. Such mechanisms require additional mass that can result in undesirable dynamic modes. OSAM would eliminate these undesirable effects by using simple joining methods such as the electron beam welding methods developed at LaRC [5]. The extra degrees

*Research Engineer, Autonomous Integrated Systems Research Branch, Mail Stop 233.
†Computer Engineer, Flight Software Systems Branch, Mail Stop 472.
‡Research Engineer, Structural Mechanics and Concepts Branch, Mail Stop 190.
§Research Engineer, Vehicle Analysis Branch, Mail Stop 451.

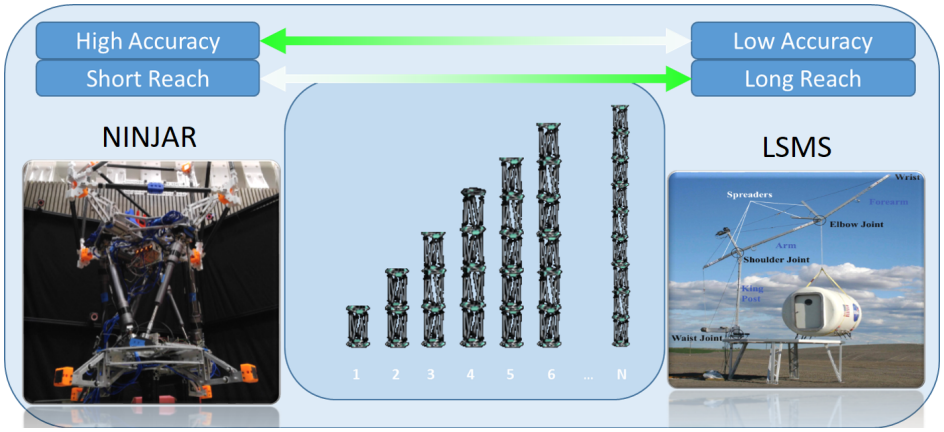


Fig. 1 Visualization of the tradeoff between high precision and long reach manipulators that can be conducted using Assemblers. The NASA Intelligent Jigging and Assembly Robot (NINJAR) represents a short range, high precision robot, and the Lightweight Surface Manipulation Systems (LSMS) represents a long reach robot with lower payload precision capability [6]. Both of these functions can be created with Assemblers, and a continuum of intermediate configurations between the two can be used to tailor the manipulator to specific applications.

of freedom required for deployment are shifted from the structure itself to an assembly tool that can be reused. This paradigm introduces advantages over the conventional deployment approach including serviceability, expandability, packing efficiency, and modularity. The use of upgradeable and repairable assembly architectures will enable rapid updates, greatly extending the typical 5 to 15-year mission life for in-space structures. For example, components that have degraded due to radiation or micrometeroid impacts could be replaced on an as-needed basis. Furthermore, space structure mass and volume will be liberated from the constraints imposed by a single launch vehicle.

Most existing OSAM concepts rely on mission-specific manipulators that are typically long-reach. These manipulators are well-suited for gross movement tasks such as transporting materials and components across a worksite. However, many assembly tasks also require high-precision manipulation to align components for joining. While long-reach manipulators may be an important part of the toolbox to perform OSAM tasks, a truly practical concept of operations must involve cooperation between long-reach and smaller, high-precision manipulators and tools. Assemblers can be used to conduct a tradeoff between precision and reach. The manipulator can be optimized for a specific application without additional non-recurring engineering efforts. This tradeoff is visualized in figure 1.

In the near term, OSAM can enable or simplify many missions and technologies, including large communications antennae, large shields for radiation protection, telescopes such as the Large UV-Optical-IR Surveyor or the one detailed in the NASA in-Space Assembled Telescope Study [7], megawatt solar arrays, and large, lightweight structures for artificial gravity [8].

II. Concept of Operations

The Assemblers modular system enables many concepts for on-orbit and surface construction and maintenance. The current Assemblers project focuses on the concept of lunar surface solar array construction for human pre-arrival site preparation. The concept of operations is depicted in figure 2.

An example mission concept is described as six acts. The first act, work site preparation, includes unloading from a lander or series of landers. Upon unloading, a task management system would analyze the overall mission given the layout of the site, location of manipulators and materials, and how the site has been staged. The second act is the foundation preparation for the structure. In this act, manipulator configurations are determined, tasks are allocated to the manipulators, material is moved from staging area to foundation area, and the foundation is set up and inspected. The third act is then the structural assembly including component jigging, joining, and inspection. This act also contains tool change tasks and possible manipulator reconfiguration as directed by the task manager. Act four entails error detection and correction and can be performed in parallel with act three. Act four also produces situation reports as the Assemblers build and inspect. Additionally, the task manager may re-plan depending on the results of inspection steps. The fifth act is task switching. Once the backbone structure is assembled, the manipulators would again reconfigure and

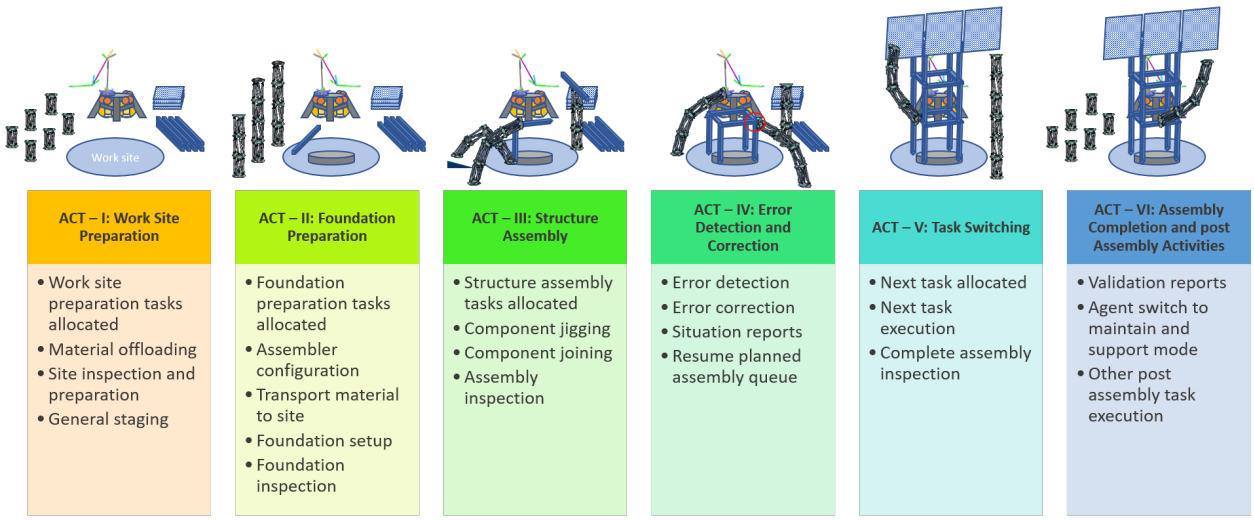


Fig. 2 Concept of operations storyboard

swap end-effectors to support the addition of components such as power boxes, wiring, and solar panels. Complete assembly inspection is also carried out in this act. Finally in act six, the system is switched into a maintenance and support mode, and other post-assembly tasks are executed.

III. Mechanical and Electrical Systems

Mechanical and electrical systems of Assemblers including the SP modules, modular connector, and end-effectors will be described in this section. While the modular SP system is intended to allow reconfiguration for different tasks, a point design for building a solar power tower was created to demonstrate capabilities, as described above. Additionally, the assembly test articles and an assembly site foundation structure, are also described in this section, in greater detail. The end-effectors were designed for the particular reference mission. The demonstrated assembly case will utilize eight SP modules that may be configured into four, three, two, or single unit manipulators, but the configuration of these in the assembly site will ultimately be decided by an autonomous task management system (section IV.B). The SPs themselves will demonstrate fine manipulation, component joining, and tool use tasks for the autonomous assembly of a vertical solar array tower.

A. Stewart Platform SP-01

The SP module presented herein, referred to as SP-01, is derived from an earlier design described in [5]. This earlier design was a proof of concept prototype using light weight plastic and composite components with four stacked SPs that shared plate components with no modular connections. The SP-01 improves upon this by dividing the SP sections into separate modules and using more powerful linear actuators, stronger metallic structural components, doubled end plates with box structures, a central column for power and data transfer between the end plates, reel based wire storage, and powerful on-board computer systems augmented with internal sensors. Additionally, commercial-off-the-shelf (COTS) u-joints coupled with thrust bearing actuator mounts were chosen over the ball-joints of the previous design. Each SP-01 module is a 6-DOF manipulator with significant computational power and sensing capabilities to create a capable robotic agent that can be combined with equally capable modules for different tasks.

Figure 3 depicts the current design iteration of the SP-01 where modularity, range of motion, and lifting power are emphasized. The SP-01 design measures 0.45 m in diameter and 0.78 m in height and can extend to 1.01 m and rotate the top plate to 45° from the vertical axis. The SP-01 module consists of a bottom plate, top plate, central column, modular connector, wire reels, electrical housing, and six linear actuators connected to u-joints and thrust bearings. The doubled top and bottom plates are configured into a box to increase plate bending stiffness, create a protective housing for the electrical components, and support the modular connectors. The doubled plates are separated by 2-inch steel stand-offs that transfer loads imparted through the linear actuators directly from the thrust bearings and into the outer-most plates to allow efficient load transfer between modules. Static analyses of loads in various configurations

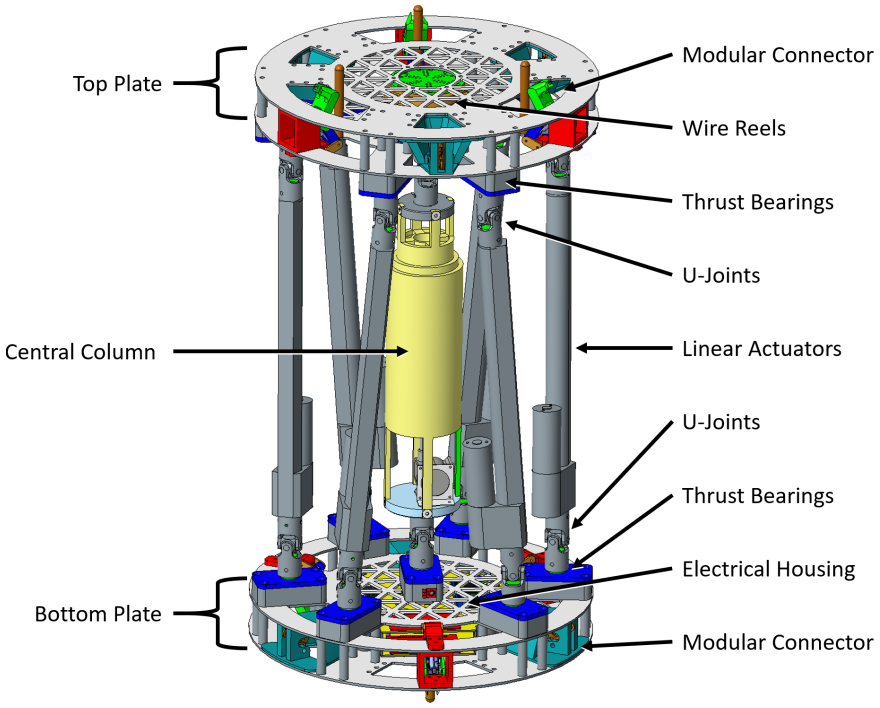


Fig. 3 Assemblers Stewart platform module, SP-01

have been performed to refine the plate design and determine range of motion. The purpose of the central column is to house power and data cables connecting the top and bottom plates. The central column is composed of three telescoping tubes connected to the two plates with the same thrust bearing and u-joint combination as the linear actuators to allow passive alignment between the center of the top and bottom plates. The thrust bearing and u-joint combination provides three degrees of rotational freedom. A COTS u-joint with a 22 mm inner diameter, which is large enough to fit over both ends of the linear actuator, is adapted to the linear actuator and the thrust bearings by custom adapters.

The SP-01 design uses COTS actuators, sensors, and computer systems to enable both accurate state estimation of each SP and the positions of the truss and joining elements that will be maneuvered for the assembly tasks. These low cost components were chosen to reduce the cost of the system to enable building multiple modules for testing. The linear actuators used in the SP-01 are Firgelli feedback rod linear actuators (model FA-PO-240-12-12) operating at 12 V, capable of producing 890 N of dynamic force, actuating a 0.3 m stroke, and providing potentiometer feedback for absolute positioning. The linear actuators are complemented by the sensor and computer systems located in the electrical housing. An overview of the sensor and computer types and locations are depicted in figure 4. High and low level computing functions of the SP-01 are distributed between a Jetson Nano performing data analysis and path planning, a Teensy microcontroller commanding the linear actuators, and the motor drivers executing linear actuator commands. Sensor inputs into the Jetson Nano include an inertial measurement unit (IMU), linear actuator feedback, a string potentiometer reporting plate distances, and camera data from the end-effector. As a stack of SPs increases in number, so does the available on-board computational power. Notably, the SP directly connected to the end-effector will be tasked with processing camera data.

Details of the electrical power bus and how the linear actuators are powered are provided in figure 5. Total potential power draw for a single SP module with linear actuators at maximum load is 374 W. Power from the 24 V bus is stepped down to 12 V to power the linear actuators, 5 V to power the on-board computers, and 3.3 V lines to power data connections. At full load, the linear actuators, which are the main power draw of the module, each require 5 A at 12 V (60 W) to function. Primary power for a stack of SPs is drawn from a standard 15 A wall outlet, through a 24 V, 1800 W power supply, and supplied to the main bus of the SP stack. The 24 V bus is connected between each SP through the central column and the SP to SP electrical connections. A switching circuit between each SP determines if on-board battery power or wall power is used. To reduce the weight of SP-01, the battery system is only able to power the electronics and linear actuators in light load cases or for a limited time. Wall power is required for the full load

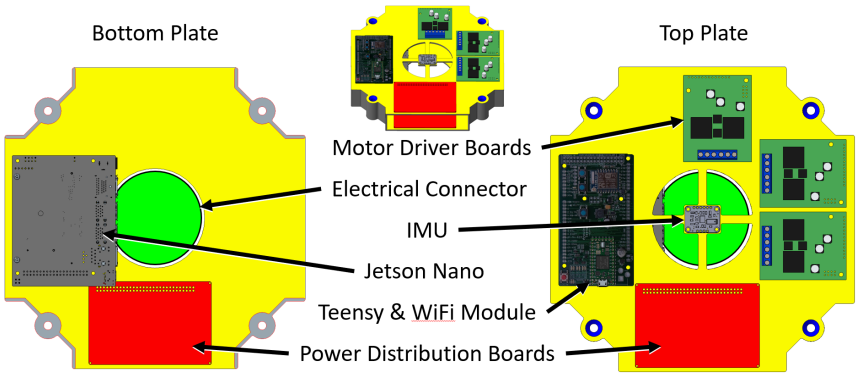


Fig. 4 Electrical housing schematic with sensor and computer type and locations

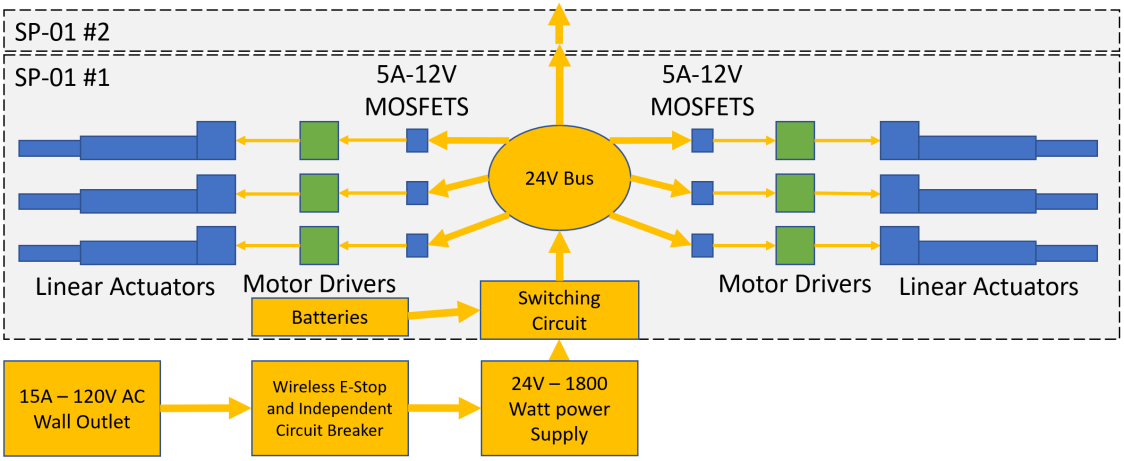


Fig. 5 24 V power bus and 12 V linear actuator power distribution

capability of SP-01 for the intended duration.

B. Modular Connectors

A modular connection system serves as a common interface between SP modules, the base connection, and any end-effectors. An overview of the modular connector is depicted in figure 6 where it is located on both the top and bottom plates of the SP-01. The modular connector both mechanically and electrically joins two modules when locking latches are engaged. Located around the perimeter of the outer most SP plates are three male latches and three female latches that alternate every 60°. The male latches are an active component with a four-bar linkage that is actuated with a PQ-12 Actuonix micro linear actuator to extend a bar. A 50 mm long guide pin is co-located with the male latches while a guide receptacle is collocated with the female latches. The female latch is fixed between the two plates and houses a COTS rotary latch.

The operation for the modular connection is shown in figure 7. There are three main steps of the connection process for two modular connectors: approach, align, and latch. During the approach phase, the SP plates are shown to be greater than 50 mm away from one another and with a 5° misalignment. The connection requires the two SPs to maneuver the plates to be in contact and clocked at 60° from one another with their primary linear actuators. The alignment phase occurs within 25 mm when the guide pins and the guide receptacles are close enough to contact. The tolerance for the guide features is +/-5° in the vertical axis of the plates, and 15 mm radially. In the latching phase, the guide pins depress contact indicators to signal that the SP plates are aligned and ready for latching. During latching, the three male latches on each SP will actuate to rotate into an extended position to engage the corresponding female rotary latches. The male latch mechanism will lock against a stop feature and apply pre-load to the joint while the female rotary latch locks into a closed position. Connections between SPs and the base mounting and end-effectors will follow the same process.

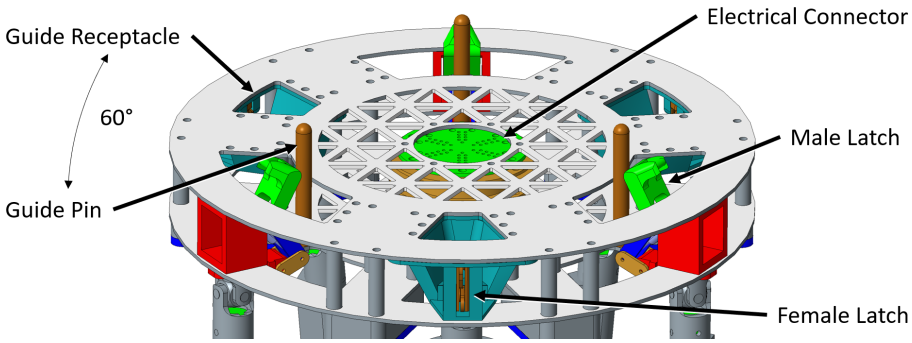


Fig. 6 Modular connector overview (top plate)

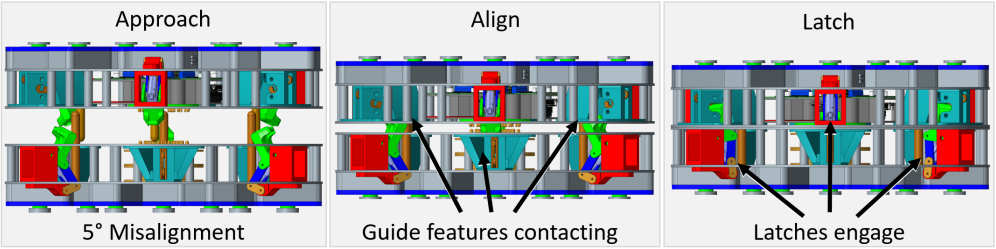


Fig. 7 Modular connection Process with 5° axial misalignment (side view)

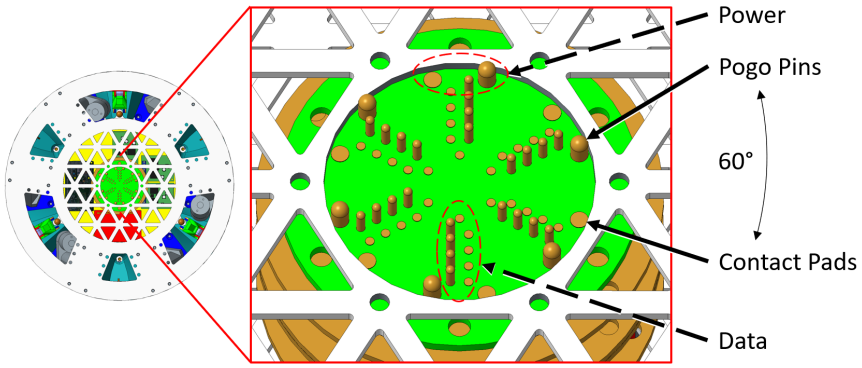


Fig. 8 Electrical connection detail

Details of the electrical connection portion of the modular connector are illustrated in figure 8. The electrical connections provided through the modular connector are power, ground, and two USB data connections. The USB connections allow direct communication between on-board computers and the end-effector electronics. There are a total of six sets of these data and power connections that utilize pogo pins and contact pad pairs. The electrical connectors are mirrored between SPs and the 60° clocking aligns with the same clocking as the mechanical connection. Contact between the pogo pins and contact pads is maintained by force provided by the perimeter latch mechanisms.

C. Assembly Site Foundation Structure

The assembly site foundation structure is ground support equipment that provides a stable base for SP stacks as well as necessary structure for assembling test articles. An overview of the foundation design is illustrated in figure 9.

The primary structure is composed of a central foundation for the truss assembly surrounded by three C-channel stiffened rails each with adjustable SP bases. The SP bases each provide a bolt pattern matching the SP plates for a fixed connection but are also able to host a modular connector. Different base configurations are achievable by manually rotating C-channel rails relative to the foundation and sliding the bases along their respective rails. The rails and base

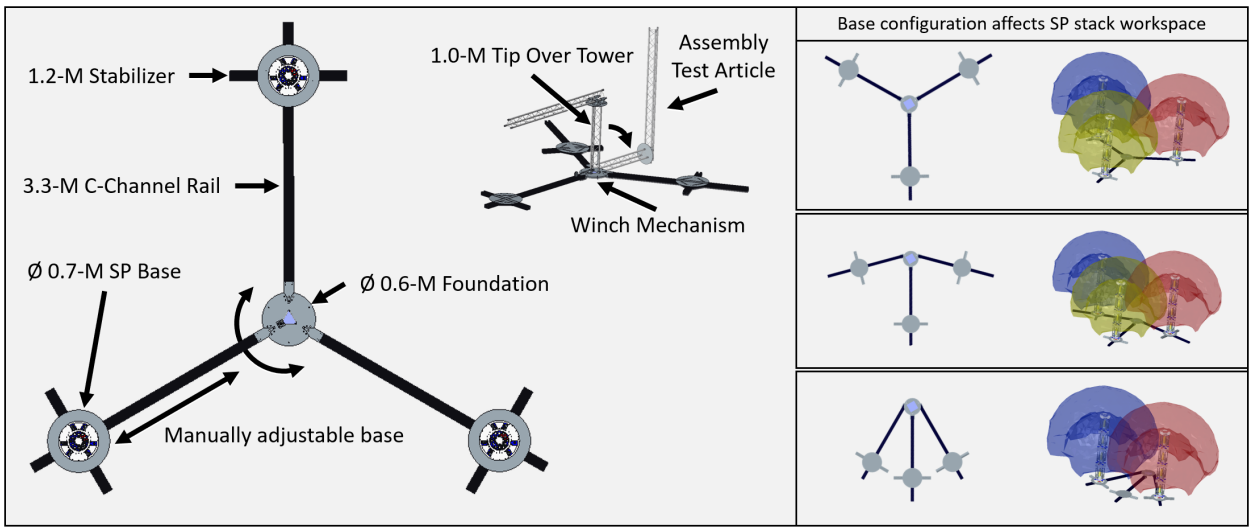


Fig. 9 Assembly site foundation structure and resulting manipulator workspaces

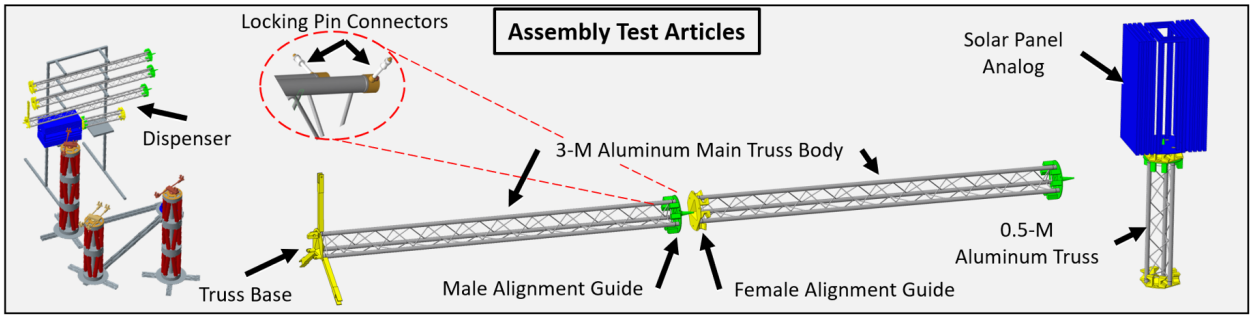


Fig. 10 Overview of assembly test articles for the low fidelity vertical solar array tower

positions would be fixed during demonstration execution and the optimal configuration will be determined by the task management system. The manipulator workspaces for different configurations are also presented in figure 9 as blue, red, and yellow volumes. The manipulator workspaces are dependent on stack size and the base configuration and are critical to consider for demonstration execution. An important feature to accommodate the SP stack workspace is the 1.0 m tip over tower located on the foundation. The tip over tower provides a raised assembly platform that is more accessible to the taller SP stacks and also provides a winch based system for standing up a horizontally assembled tower.

D. Assembly Test Articles

The assembly test articles provide a low fidelity but functional analogue for a vertical solar array tower to demonstrate precision joining using Assemblers. They will be assembled into a 10 m tower with a deployable 6 m vertical solar array. This represents a viable method for power infrastructure at the lunar south pole. Details of the assembly test articles are illustrated in figure 10.

The test article is composed of three major components: The solar panel analogue (SPA), main truss body, and truss base. The SPA serves as a mass analogue for the deployable solar array while in its stowed state and will not actually deploy during the construction demonstration. The main truss body will be built from three identical 3 m long equilateral triangle truss segments with 0.2 m sides. The truss segments are COTS with minor modifications to simplify handling and joining by robotic tools. The joining is facilitated by male and female alignment guides that properly position the truss segments to allow locking pin connectors to create the connection. The truss base stabilizes the structure while on the ground. The SPA is pre-attached to a 0.5 m truss segment, which allows the module to utilize the truss joining and handling interface.

Figure 11 depicts the demonstration procedure. The demonstration will begin with the assembly site staged with

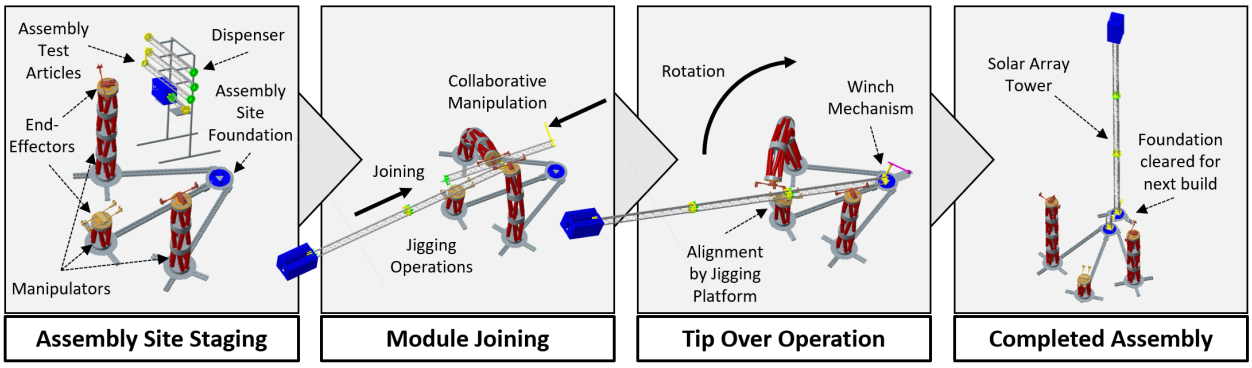


Fig. 11 Overview of assembly process for the low fidelity vertical solar array tower

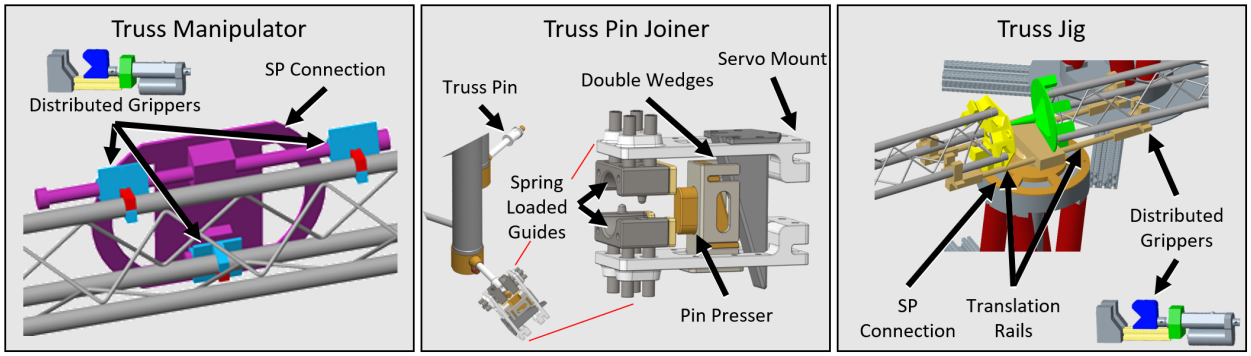


Fig. 12 Overview of end-effectors created for the Assemblers system

the foundation, manipulators, and a dispenser holding the test articles already positioned. During the construction demonstration, the test articles for the solar array tower will be assembled on the tip over tower using Assemblers of various sizes for both gross (possibly collaborative) manipulation and precision jigging. Once the tower is assembled, the Assemblers system will reorient it vertically. The final step of the assembly process is positioning, anchoring, and deployment of the solar array. This will be accomplished by either the manipulators, or the winch mechanism on the tip over tower.

E. End-Effectors

Details of the three end-effectors planned to be used with the assembly test articles are illustrated in figure 12. All end-effectors will connect to the end SP module through the modular connector. The truss manipulator is dedicated to grasping the truss segments with three or more powered grippers. The grippers will be independently actuated by linear actuators and distributed to react to moments during manipulation. Ideally, the end-effector will grasp truss segments close to the truss's center-of-mass (COM) to minimize moments imparted into the manipulator. However, collaborative manipulation with other agents may require multiple grasping interfaces.

The truss pin joiner tool is the active part of a larger end-effector and will be used to lock two aligned truss segments together with three pins each. The tool is a modification of the COTS pin lock system used on the truss segment product. The pins will be pre-positioned on the truss segments, and each will have a spring-loaded push-to-release button located on the exposed end. The truss pin joiner has two main components: the pin gripper and the pin presser. Both components of the tool are actuated by a single servo motor as the servo mount translates on a rail. The pin gripper is engaged first as the servo actuates to close two spring-loaded guides over the truss pin. As the servo closes the servo mount to an end-stop, the pin presser component is engaged by two wedged surfaces that force the pin pusher to compress the button on the truss pin. Once the button is pressed, the truss pin is now free to be pushed into the truss-to-truss interface. The pin gripper will then release itself from the truss pin to complete the locking procedure.

The truss jig will temporarily hold the trusses together while a truss pin joiner end-effector completes the connection. While on the truss jig, two truss segments are held by the distributed grippers located on translating rails. These are also

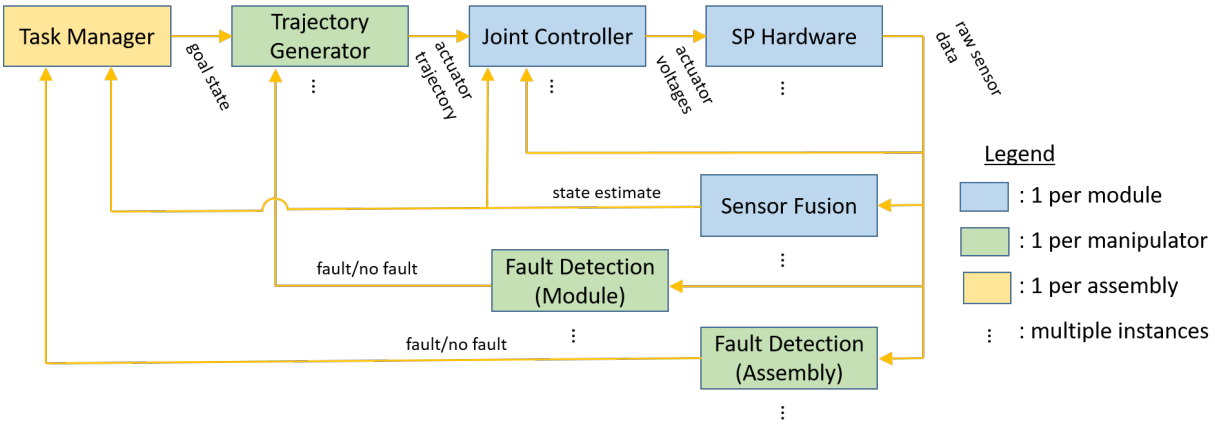


Fig. 13 Control architecture block diagram

supported by either the tip over tower or a manipulator. The translating rails are capable of performing final alignment on truss segments for the pin locking system or for shifting the grip point on a single truss segment.

Not pictured is the camera system that is integrated with each end-effector and used for pose estimation of the test articles. A Microsoft Azure Kinect camera was chosen for the end-effectors because of the high resolution color sensor (12-megapixel), embedded IMU, and well-supported developer tools. Integration of the camera is eased by compatibility with the 12-Volt power and USB data systems of the SP modules. The Azure Kinect also provides a 1-megapixel depth sensor.

IV. Dynamics, Control, and Sensing Systems

A. Control Architecture

The control architecture for the Assemblers system is shown in the block diagram in figure 13. There are six software systems that control Assemblers: the task manager, trajectory generator, joint controller, sensor fusion system, assembly fault detection system, and module fault detection system. Each of these is described in the following subsections except for the module fault detection which is still in early development. The joint controller and sensor fusion systems run onboard each SP module. An assembly process may use multiple manipulators, and each manipulator will have its own trajectory generator and fault detection systems. A single task manager is used to assign tasks to each of the manipulators in an assembly process, as well as select the number of modules in each manipulator.

B. Task Manager

The task manager serves two functions: 1) determine the optimal number of modules for each manipulator involved in the assembly process, and 2) assign tasks to each of the manipulators involved in the assembly process. The task manager will have access to a simulation of the assembly which can be used to optimize its assembly plan with respect to a given criterion. Two possible criteria to minimize are build time and energy used. Reinforcement learning is being considered as an approximate optimization method for scenarios with large decision spaces [9].

C. Trajectory Generator

1. Path Planning

The trajectory generator takes as input a 6 DOF goal pose for the manipulator end-effector, and outputs trajectories for the lengths of each linear actuator for all SP modules in the manipulator. The goal pose represents the task allocated by the task manager. The manipulator must move its end-effector or payload to that pose to perform the assigned assembly task.

The trajectory generator uses the optimizing version of the Rapidly-exploring Random Trees algorithm (RRT*) [10]. The RRT* algorithm samples points in the actuator length space (configuration space) and checks the validity of the

resulting manipulator pose. Checking the validity relies on both the module and manipulator level forward kinematics (FK) as well as a collision checking algorithm. The collision-checking algorithm employed is the Gilbert-Johnson-Keerthi (GJK) algorithm [11], which can efficiently check for collisions between 3D objects represented as meshes.

Pseudocode for the implementation of RRT* used (in part adapted from [12]) is presented in algorithm 1. RRT* finds a valid path once the goal state, x_{goal} is present in the tree, T. Once this criterion is met, further nodes added to the tree serve to optimize a computable cost function, f, of the path.

The configuration space, $\mathcal{X}$, of an Assemblers manipulator is large ($6n$, where n is the number of modules in the manipulator). Thus the chances of randomly sampling the goal state are low, so RRT* may take many iterations to find a valid path. To encourage the algorithm to find valid paths more quickly, samples are biased towards the goal state with a probability, p_{goal}, of directly sampling the goal state. This requires knowledge of the goal state in configuration space.

Algorithm 1 RRT* algorithm for path planning

function RRT*($x_{start}, p_{goal}, x_{goal}, \delta_{nom}$)
  initialize search tree T with x_{start}
  success ← False
  while T is less than the maximum tree size **do**
    dice ← sample from $U(0, 1)$
    if dice < p_{goal} **then**
      $x_{samp} \leftarrow x_{goal}$
    else
      $x_{samp} \leftarrow$ sample from $\mathcal{X}$
    $x_{nearest} \leftarrow$ nearest node in T to x_{samp}
    $\delta \leftarrow \min(\delta_{nom}, ||x_{samp} - x_{nearest}||)$
    $x_{new} \leftarrow x_{nearest} + \delta \frac{x_{samp} - x_{nearest}}{||x_{samp} - x_{nearest}||}$
    if x_{new} is a valid pose **then**
      $\mathcal{X}_{near} \leftarrow$ nodes in T with distance to $x_{new} < \delta_{nom}$
      $x_{parent} \leftarrow \arg\min_{x \in \mathcal{X}_{near}} f(x, x_{new}, T)$
      add x_{new} to T with an edge from x_{parent} to x_{new}
      for $x \in \mathcal{X}_{near}$ **do**
        if $f(x_{new}, x, T) < f(\text{parent of } x, x, T)$ **then**
          parent of $x \leftarrow x_{new}$
      if success = False and x_{new} is in $\mathcal{X}_{goal}$ **then**
        success ← True
        $x_{end} \leftarrow x_{new}$
  if success = False **then**
    $x_{end} \leftarrow$ node in T with minimum distance to x_{goal}
  return success and the motion from x_{start} to x_{end}

2. Kinematics

To obtain a valid goal state in configuration space, a numerical solver is applied to minimize the error between the goal end-effector pose and the end-effector pose calculated by the manipulator-level FK as a function of SP top plate poses. The actuator lengths are calculated from the solved SP top plate poses using the module-level inverse kinematics (IK).

The FK for the manipulator are solved iteratively based on the number of SP modules. Let $x \in \mathbb{R}^{7n}$ be the vector of module states where n is the number of modules in the manipulator.

$$x = \left[\vec{p}_{B_1}^{\top}, \ldots, \vec{p}_{B_j}^{\top}, \ldots, \vec{p}_{B_n}^{\top}, q_{B_1}^{T_1\top}, \ldots, q_{B_j}^{T_j\top}, \ldots q_{B_n}^{T_n\top}\right]^{\top} \tag{1}$$

where $\vec{p}_{B_j}$ is the position vector of the top plate of module j measured in coordinate frame B_j attached to the bottom plate of module j and $q_{B_j}^{T_j}$ is the quaternion representing the rotation from B_j to coordinate frame T_j attached to the top plate of module j for $j = 1, 2, \ldots, n$. The FK algorithm is given in algorithm 2 where $\odot$ represents quaternion

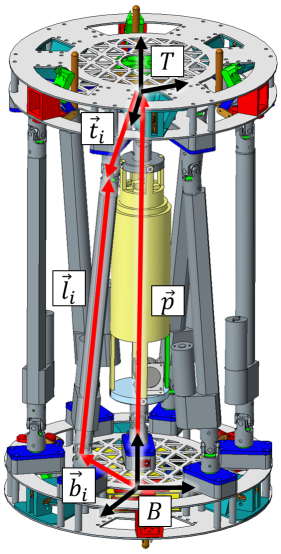


Fig. 14 Notation used for Stewart platform inverse kinematics

multiplication. It is assumed that the bottom plate of the first module is attached to a global reference frame. It is also assumed that $T_j = B_{j+1}$ for $j = 1, 2, \ldots, n-1$. $\vec{p}$ is the position vector of the top plate of module n measured in B_1 and q is the quaternion representing the rotation from B_1 to T_n. Together $\vec{p}$ and q constitute the pose of the manipulator end effector relative to the global reference frame.

Algorithm 2 Assemblers manipulator forward kinematics

function FK(x)
  Initialize quaternion $q \leftarrow \begin{bmatrix} 1 & 0 & 0 & 0 \end{bmatrix}^\top$
  Initialize position $\vec{p} \leftarrow \begin{bmatrix} 0 & 0 & 0 \end{bmatrix}^\top$
  for $j = 1, 2, \ldots, n$ **do**
    $\vec{p} \leftarrow \vec{p} + q \odot \vec{p}_{B_j} \odot q^*$
    $q \leftarrow q_{B_j}^{T_j} \odot q$
  return $\vec{p}$, q

The error function to minimize for the manipulator IK solution is

$$E(x) = (\vec{p}(x) - \vec{p}_{des})^\top (\vec{p}(x) - \vec{p}_{des}) + (Re(q(x) \odot q_{des}^*) - 1)^2 \tag{2}$$

where $\vec{p}(x)$ and $q(x)$ are the end-effector position and quaternion respectively returned by $FK(x)$ in algorithm 2, $\vec{p}_{des}$ and q_{des} are the position and quaternion of the goal pose respectively. $E(x)$ is minimized numerically subject to the constraints that each quaternion in x has unit magnitude.

The manipulator kinematics relate the module poses to the end-effector pose. To specify the goal state for the RRT* algorithm, the end-effector pose must ultimately be related to the actuator lengths of each module. Module poses are related to actuator lengths via the SP kinematics. For the following discussion, the subscript j is dropped since these kinematics apply to all SP modules. Let $\vec{b}_i$ be the position vector of the attachment point of actuator i to the SP's bottom plate in coordinate frame B and $\vec{t}_i$ be the position vector of the attachment point of actuator i to the SP's top plate in coordinate frame T. Let $\vec{l}_i$ be a vector aligned with actuator i of magnitude L_i where L_i is the length of actuator i. Figure 14 shows how this notation relates to the physical SP.

From figure 14,

$$\begin{aligned} \vec{p}_B &= \vec{b}_{i_B} + \vec{l}_{i_B} - \vec{t}_{i_B} \\ &= \vec{b}_{i_B} + \vec{l}_{i_B} - q_B^{T*} \odot \vec{t}_{i_T} \odot q_B^T \end{aligned} \tag{3}$$

Solving (3) for $\vec{l}_{i_B}$ yields

$$\vec{l}_{i_B} = \vec{p}_B - \vec{b}_{i_B} + q_B^{T^*} \odot \vec{t}_{i_T} \odot q_B^T \tag{4}$$

The actuator length, L_i is the magnitude of the vector $\vec{l}_i$, so the IK function can be written as

$$L_i(\vec{p}_B, q_B^T) = ||\vec{p}_B - \vec{b}_{i_B} + q_B^{T^*} \odot \vec{t}_{i_T} \odot q_B^T|| \tag{5}$$

With the combination of manipulator and module kinematics functions/solvers, a set of linear actuator lengths corresponding to an end-effector pose can be obtained to use as the goal state of RRT* in algorithm 1.

3. Collision Checking

The collision checking algorithm employed for trajectory generation is the Gilbert-Johnson-Keerthi (GJK) algorithm [11]. The GJK algorithm relies on concept of the Minkowski difference between sets representing the two objects being checked. The Minkowski difference is the set of points comprised of subtracting every point in the first object from every point in the second object. If the Minkowski difference contains the origin, then the objects share at least one point and thus intersect.

GJK works by maintaining a simplex of up to 4 points within the Minkowski difference. The simplex is moved around until it either encloses the origin which indicates a collision or the origin cannot be reached which indicates no collision.

The algorithm uses one of three subroutines for updating the simplex based on the number of points it contains: line_case, triangle_case, or tetrahedron_case. See appendix A for details of each of these subroutines. Each subroutine returns a search direction, d. The furthest point in direction, d, contained in the Minkowski difference is added to the simplex. If the new point does not go past origin, there is no collision, and if the origin is enclosed in tetrahedron, there is a collision. The algorithm iterates until one of these conditions is met. Pseudocode for the GJK algorithm and its subroutines is given in appendix A.

4. Trajectory Generation

The path produced by RRT* is a list of actuator positions. The trajectory is created from this list of positions by fitting a polynomial spline through each point in the list. The spline can then be differentiated to obtain speed and acceleration commands for each actuator. The acceleration command in conjunction with the mass properties of the payload of each SP can be used to obtain the load through the actuator. The travel time for the trajectory is scaled such that speed and load constraints of the actuators are not violated.

The polynomial splines used are continuous up to the third derivative (jerk). The initial and final velocity, acceleration, and jerk of the trajectory are all constrained to zero. With the given initial and final positions, there are a total of eight constraints, so polynomial segments with eight parameters are used for the spline. Polynomial segment i is represented as

$$r_i(\tau) = \theta_0 + \sum_{j=1}^{7} \theta_{j_i} \tau^j \tag{6}$$

where r_i is the polynomial value, $\tau \in [0, 1]$ is the independent variable, j is the coefficient index, and θ_{j_i} are the polynomial coefficients.

For each intermediate point in the path list, the following additional constraints must hold:

$$\begin{aligned} r_i(0) &= r_{0_i} \\ r_i(1) &= r_{f_i} \\ v_i(1) &= v_{i+1}(0) \\ a_i(1) &= a_{i+1}(0) \\ j_i(1) &= j_{i+1}(0) \end{aligned} \tag{7}$$

where r_{0_i} is the initial position of segment i, r_{f_i} is the final position of segment i, and v_i, a_i, and j_i are the first, second, and third derivatives of (6) respectively.

Let

$$
\begin{aligned}
\vec{\theta}_i &= \begin{bmatrix} \theta_{0_i} & \theta_{1_i} & \theta_{2_i} & \theta_{3_i} & \theta_{4_i} & \theta_{5_i} & \theta_{6_i} & \theta_{7_i} \end{bmatrix}^\top \\
\vec{\rho}_{r_0} &= \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}^\top \\
\vec{\rho}_{v_0} &= \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}^\top \\
\vec{\rho}_{a_0} &= \begin{bmatrix} 0 & 0 & 2 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}^\top \\
\vec{\rho}_{j_0} &= \begin{bmatrix} 0 & 0 & 0 & 12 & 0 & 0 & 0 & 0 \end{bmatrix}^\top \\
\vec{\rho}_{r_1} &= \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}^\top \\
\vec{\rho}_{v_1} &= \begin{bmatrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 \end{bmatrix}^\top \\
\vec{\rho}_{a_1} &= \begin{bmatrix} 0 & 0 & 2 & 6 & 12 & 20 & 20 & 42 \end{bmatrix}^\top \\
\vec{\rho}_{j_1} &= \begin{bmatrix} 0 & 0 & 0 & 12 & 24 & 60 & 120 & 240 \end{bmatrix}^\top
\end{aligned}
\tag{8}
$$

Using the definitions in (8),

$$
\begin{aligned}
r_i(0) &= \vec{\rho}_{r_0}^\top \vec{\theta}_i, \quad & r_i(1) &= \vec{\rho}_{r_1}^\top \vec{\theta}_i, \\
v_i(0) &= \vec{\rho}_{v_0}^\top \vec{\theta}_i, \quad & v_i(1) &= \vec{\rho}_{v_1}^\top \vec{\theta}_i, \\
a_i(0) &= \vec{\rho}_{a_0}^\top \vec{\theta}_i, \quad & a_i(1) &= \vec{\rho}_{a_1}^\top \vec{\theta}_i, \\
j_i(0) &= \vec{\rho}_{j_0}^\top \vec{\theta}_i, \quad & j_i(1) &= \vec{\rho}_{j_1}^\top \vec{\theta}_i
\end{aligned}
\tag{9}
$$

With no intermediate path points, the eight constraints specified by the initial and final conditions of the trajectory must be satisfied. Each intermediate point adds the five constraints of (7). The total number of spline parameters must be at least equal to the total number of constraints. Then the required number of parameters per spline segment, l, is

$$
l = ceil\left(\frac{5(n-1)+8}{n}\right) \tag{10}
$$

where n is the number of spline segments. Thus all of the vectors in (8) can be truncated to their first l elements.

All of the constraint equations can be written in a matrix form as

$$
\begin{bmatrix} r_{0_0} \\ \vec{r}_{WP} \\ \vec{r}_{WP} \\ r_{f_n} \\ [0]_{3(n+1)\times 1} \end{bmatrix} = M \begin{bmatrix} \vec{\theta}_0 \\ \vdots \\ \vec{\theta}_n \end{bmatrix} \tag{11}
$$

where $\vec{r}_{WP} = \begin{bmatrix} r_{f_0} & \dots & r_{f_{n-1}} \end{bmatrix}$ and

$$M = \begin{bmatrix} \mathbb{I}_n \otimes \vec{\rho}_{r_0}^\top \\ \mathbb{I}_n \otimes \vec{\rho}_{r_f}^\top \\ \begin{bmatrix} \vec{\rho}_{v_0}^\top & [0]_{1\times l(n-1)} \end{bmatrix} \\ \mathbb{I}_n \otimes \vec{\rho}_{v_f}^\top + \begin{bmatrix} \begin{bmatrix} [0]_{(n-1)\times l} & \mathbb{I}_{n-1} \end{bmatrix} \\ [0]_{1\times n} \end{bmatrix} \otimes -\vec{\rho}_{v_0}^\top \\ \begin{bmatrix} \vec{\rho}_{a_0}^\top & [0]_{1\times l(n-1)} \end{bmatrix} \\ \mathbb{I}_n \otimes \vec{\rho}_{a_f}^\top + \begin{bmatrix} \begin{bmatrix} [0]_{(n-1)\times l} & \mathbb{I}_{n-1} \end{bmatrix} \\ [0]_{1\times n} \end{bmatrix} \otimes -\vec{\rho}_{a_0}^\top \\ \begin{bmatrix} \vec{\rho}_{j_0}^\top & [0]_{1\times l(n-1)} \end{bmatrix} \\ \mathbb{I}_n \otimes \vec{\rho}_{j_f}^\top + \begin{bmatrix} \begin{bmatrix} [0]_{(n-1)\times l} & \mathbb{I}_{n-1} \end{bmatrix} \\ [0]_{1\times n} \end{bmatrix} \otimes -\vec{\rho}_{j_0}^\top \end{bmatrix} \tag{12}$$

where $\otimes$ denotes the Kronecker product. Then a feasible solution for the spline parameters can be found using the Moore-Penrose pseudoinverse

$$\begin{bmatrix} \vec{\theta}_0 \\ \vdots \\ \vec{\theta}_n \end{bmatrix} = M^+ \begin{bmatrix} r_{0_0} \\ \vec{r}_{WP} \\ \vec{r}_{WP} \\ r_{f_n} \\ [0]_{3(n+1)\times 1} \end{bmatrix} \tag{13}$$

The pseudoinverse is used to handle cases where $nl \geq 5(n-1)+8$.

The trajectory specified by the polynomial spline uses a normalized time for the independent variable. To convert to real time, the physical properties of the actuators must be considered. The actuators have a maximum speed, v_{max}, as well as a maximum load, F_{max}.

First the trajectory is scaled to address the speed limit. A scaling parameter α_v is calculated as

$$\alpha_v = \frac{v_{max}}{\max_i(||v_i(\tau)||_{\mathcal{L}_\infty})} \tag{14}$$

α_v is then used to scale each spline parameter according to

$$\theta_{j_i} \leftarrow \alpha_v^j \theta_{j_i} \tag{15}$$

for all i, j. The updated normalized time is also scaled to an intermediate variable, $\bar{\tau} = \tau/\alpha_v$.

Next, the trajectory is scaled to meet actuator load limits. Actuator loads throughout the trajectory are calculated using the SP kinematics, the acceleration of the trajectory, and the mass properties of the SP's payload which includes all downstream SPs in the manipulator, the manipulator end-effector, and any objects currently held by the end-effector. Let $F_i(\bar{\tau})$ be the load through actuator i over the intermediate variable, $\bar{\tau}$. The trajectory is only scaled if $\max_i(||F_i(\bar{\tau})||_\infty) > F_{max}$. The load scaling parameter, α_F is calculated according to

$$\alpha_F = \sqrt{\frac{F_{max}}{\max_i(||F_i(\bar{\tau})||_\infty)}} \tag{16}$$

α_F is then used to scale each spline parameter according to

$$\theta_{j_i} \leftarrow \alpha_F^j \theta_{j_i} \tag{17}$$

for all i, j. The real time for the trajectory is then $t = \bar{\tau}/\alpha_v$.

D. Joint Controller

The joint controller receives position and speed commands for each actuator in the module, as well as the expected load through each actuator required to follow the trajectory specified by the trajectory generator. The position command is used to generate a feedback control signal using a proportional controller, and the speed command and load are used to generate a feedforward signal using calibration curves for the actuator. The control signals generated by the joint controller are the PWM signals applied directly to each actuator.

The linear actuator control law is

$$u_{sat} = clip(u, -1, 1) \tag{18}$$

where

$$u = u_{fb} + u_{ff} \tag{19}$$

where

$$u_{fb} = K_p(L_{cmd} - L) \tag{20}$$

where K_p is a proportional gain, L_{cmd} is the commanded actuator length, and L is the measured actuator length, and

$$u_{ff} = \frac{v_{cmd}}{K_V K_F V_{nom}} \tag{21}$$

where v_{cmd} is the commanded actuator speed,

$$K_V = \frac{v_{nom}}{V_{nom}} \tag{22}$$

where v_{nom} is the actuator's speed at its nominal voltage, V_{nom}, and

$$K_F = 1 + KF \tag{23}$$

where K is the slope of the normalized speed vs. load curve for the actuator and F is the load through the actuator.

E. Sensor Fusion

A sensor fusion method is developed which takes information from the control inputs and sensors on each SP to generate estimates of pose of the SP's top plate relative to its bottom plate. The sensor fusion system uses an Extended Kalman Filter (EKF) to compute estimates in terms of conditional probability distributions. In order to test the sensor fusion process, simulations for the onboard sensors are also developed. These include an inertial measurement unit (IMU) and a string potentiometer. This section will lay out the dynamics and sensor models as well as the EKF algorithm.

1. Dynamics Models

As shown in [13], the continuous-time relation that forms the basis of the dynamics models is given by the position, $\vec{p}$, velocity, $\vec{v}$, and acceleration, $\vec{a}$, of the SP top plate relative to its bottom plate.

$$\vec{v} = \frac{d\vec{p}}{dt}, \qquad \vec{a} = \frac{d\vec{v}}{dt} \tag{24}$$

The top plate orientation and angular velocity, $\vec{\omega}$, with respect to the bottom plate are related as

$$\frac{dq_B^T}{dt} = q_B^T \odot \frac{1}{2}\vec{\omega} = \frac{1}{2}\Omega q_B^T \tag{25}$$

where q_B^T is the quaternion representing the rotation from the coordinate frame B fixed to the bottom plate to the coordinate frame T fixed to the top plate and

$$\Omega = \begin{bmatrix} 0 & -\omega_x & -\omega_y & -\omega_z \\ \omega_x & 0 & \omega_z & -\omega_y \\ \omega_y & -\omega_z & 0 & \omega_x \\ \omega_z & \omega_y & -\omega_x & 0 \end{bmatrix}$$

with $\vec{\omega} = \begin{bmatrix}\omega_x & \omega_y & \omega_z\end{bmatrix}^\top$. To find the relations in (24), an Euler discretization can be used assuming that the acceleration is constant between samples. Then the position and velocity can be expressed as

$$\vec{p}_{t+1} = \vec{p}_t + \Delta_t \vec{v}_t + \frac{1}{2}(\Delta_t)^2 \vec{a}_t, \qquad \vec{v}_{t+1} = \vec{v}_t + \Delta_t \vec{a}_t \tag{26}$$

where Δ_t is the time-step between samples. Similarly, as shown in [14], the dynamics of the orientation can be expressed as

$$q^T_{B_{t+1}} = q^T_{B_t} \odot \exp_q\left(\frac{\Delta_t}{2}\vec{\omega}_t\right) = \Phi q^T_{B_t} \tag{27}$$

where

$$\Phi = \cos\left(\frac{\|\vec{\omega}_t \Delta_t\|}{2}\right)\mathbb{I}_4 + \frac{\sin\left(\|\vec{\omega}_t \Delta_t\|/2\right)}{\|\vec{\omega}_t \Delta_t\|/2}\Omega_t. \tag{28}$$

2. Measurement Models

This section describes the IMU and string potentiometer models which are used to test the sensor fusion platform before being implemented on the SP. The models take into account several noise factors that can potentially affect the measurement readings. The IMU models herein are based off the IMU simulations models used in MATLAB® 2020a Sensor Fusion and Tracking Toolbox™[15].

The IMU provides information from which orientation can be estimated. Given $\vec{\omega}$, $\vec{a}$, and q^T_B, a model can be developed using several types of sensor biases. The gyroscope model and accelerometer models for the IMU consider sample rate, temperature, axis misalignment, constant bias, drift, noise, measurement range, and sensor resolution.

The string potentiometer provides a distance measurement between the two SP plates which can be used in conjunction with the IMU to estimate position. The potentiometer model uses the sensor's two attachment points and the orientation of the top plate with respect to the bottom plate.

The formulation of the problem is described by

$$\|\vec{l}_p\| = \|\vec{p}_B + q^{T*}_B \odot \vec{m}_T \odot q^T_B - \vec{n}_B\| + \epsilon \tag{29}$$

where $\vec{l}_p$ is a vector pointing from the potentiometer's top plate attachment point to its bottom plate attachment point, $\vec{m}_T$, is the position of the top plate attachment point in T, $\vec{n}_B$ is the position of the bottom plate attachment point in B, and ϵ is a Gaussian noise term. Figure 15 shows a diagram which describes all the variables for the string potentiometer model.

3. Extended Kalman Filter

An EKF is used to estimate the pose of the top plate relative to the bottom plate given the sensor noise. Let the state vector be

$$\mathbf{x}_t = \begin{bmatrix}\vec{p}_t^\top & q^{T^\top}_{B_t} & \vec{v}_t^\top & \vec{a}_t^\top & \vec{\omega}_t^\top\end{bmatrix}^\top \tag{30}$$

The change in position, velocity, and orientation states can then be described in terms of velocity, acceleration, and angular velocity states, respectively using the dynamics in section IV.E.1.

The EKF uses a nonlinear state space model. Assuming the measurement noise is additive and represented by zero-mean Gaussian noise, the state space model is

$$\begin{aligned}\mathbf{x}_{t+1} &= f_t(\mathbf{x}_t, \mathbf{u}_t)\\ \mathbf{y}_t &= h_t(\mathbf{x}_t)\end{aligned} \tag{31}$$

where $\mathbf{u}_t$ is the control input comprised of commanded changes in linear actuator lengths and $\mathbf{y}_t$ is the measurement vector comprised of signals from the IMU and string potentiometer.

The EKF first makes a-priori updates to its state and covariance estimates according to

$$\begin{aligned}\hat{\mathbf{x}}_{t|t-1} &= f_t(\hat{\mathbf{x}}_{t-1|t-1}, \mathbf{u}_t)\\ P_{t|t-1} &= A_{t-1} P_{t-1|t-1} A^T_{t-1} + Q\end{aligned} \tag{32}$$

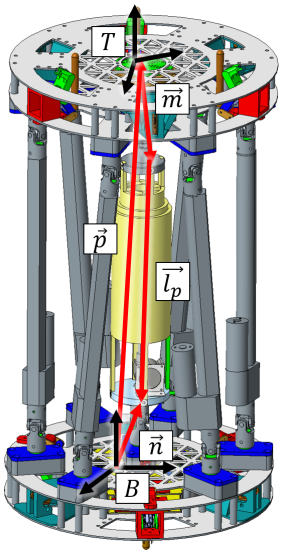


Fig. 15 Variables used for Stewart platform string potentiometer model

where A_{t-1} is the state matrix at time $t-1$ found by linearizing $f_t(\mathbf{x}_t)$,

$$A_{t-1} = \left.\frac{\partial f_t(\mathbf{x}_t, \mathbf{u}_t)}{\partial \mathbf{x}_t}\right|_{\mathbf{x}_t = \hat{\mathbf{x}}_{t-1|t-1}, \mathbf{u}_t = \mathbf{u}_{t-1}} \tag{33}$$

P_{t-1} is the state covariance, and Q is the process noise covariance. The double subscripts on $\hat{\mathbf{x}}_{t|t-1}$ and $P_{t|t-1}$ denote the state estimate and state covariance at time t given measurements up to time $t-1$, respectively.

Next, the measurement update step makes use of a linearization of the measurement model around the state estimate of the previous time step

$$\hat{\mathbf{y}}_{t|t-1} = C_t \hat{\mathbf{x}}_{t|t-1} \tag{34}$$

with $C_t = \left.\frac{\partial h_t(\mathbf{x}_t)}{\partial \mathbf{x}_t}\right|_{\mathbf{x}_t = \hat{\mathbf{x}}_{t|t-1}}$. The measurement update step is then used to update the state and covariance estimates given the sensor measurements according to

$$\hat{\mathbf{x}}_t = \hat{\mathbf{x}}_{t|t-1} + K_t(\mathbf{y}_t - \hat{\mathbf{y}}_{t|t-1}) \tag{35}$$

$$P_t = (1 - K_t C_t) P_{t|t-1} \tag{36}$$

with $K_t = P_{t|t-1} C_t^T (C_t P_{t|t-1} C_t^T + R)^{-1}$ where R is the measurement covariance matrix. Details on the Jacobians used to calculate A_{t-1} and C_t are provided in appendix B.

F. Fault Detection

The assembly fault detection system is responsible for detecting errors in the alignment of components to be assembled. Once an Assembler completes a task that involved aligning parts for assembly, a camera on the end-effector will take an image of the two components. The fault detection system takes the image as input and outputs a fault/no fault signal. If a fault is detected, a new task is created in the task manager to correct the alignment. Otherwise, the assembly process continues as planned.

The fault detection system uses deep learning to perform the fault/no fault classification. The classifier is trained using synthetic data generated from CAD models of the assembly components. Examples of the synthetic images are shown in figure 16.

The first step of the overall process for the fault detection classifier is to estimate the pose of the object not held by the robot's end-effector. This step uses the Mask Regional-Convolutional Neural Network (Mask R-CNN) algorithm to

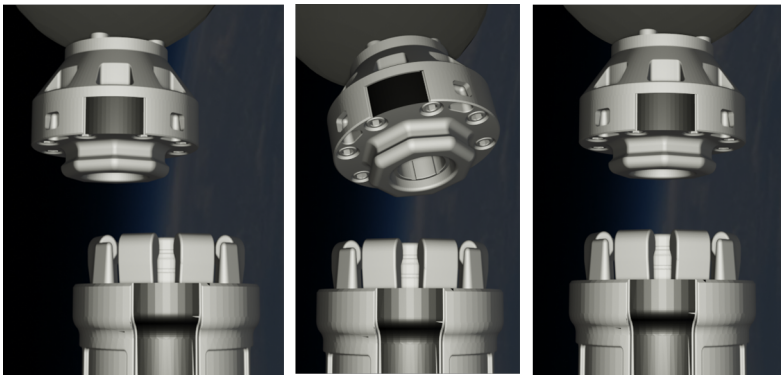


Fig. 16 Example synthetic images for training Mask R-CNN. Left: misaligned position. Center: misaligned orientation. Right: properly aligned. The CAD models used for these renderings were developed by Honeybee Robotics and are patent pending.

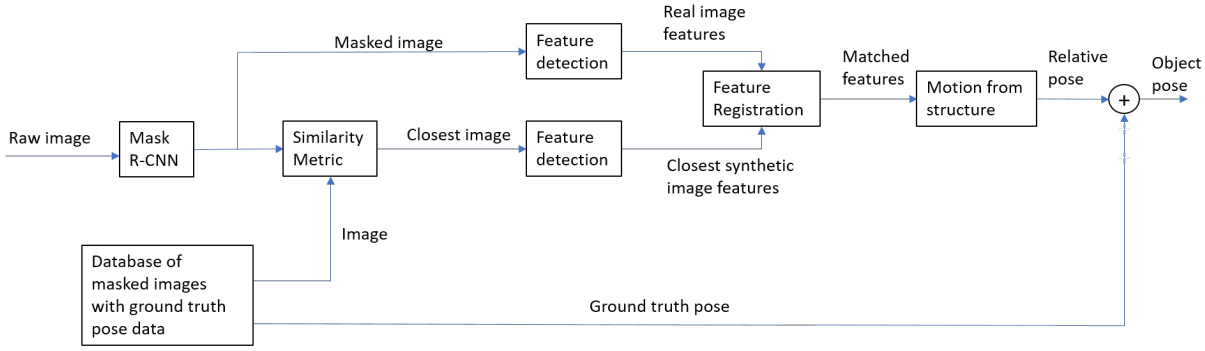


Fig. 17 Architecture diagram for pose estimation portion of fault detection system.

segment the object from the image [16]. This masked image is then compared to images from a database with known ground truth pose data using a similarity metric. Feature detection and registration are performed between the test image and the image with the best similarity score from the database. A motion from structure algorithm is applied to the pairs of matched features to produce the change in pose of the object between the two images. Finally, the ground truth data is used with the change in pose to produce an estimate of the object pose relative to the robot end-effector. This process is further described by the architecture diagram in figure 17.

Since the object held by the end-effector has a fixed pose relative to the camera on the end-effector, the relative pose of the two objects can be calculated from the previously described pose estimate. If the relative pose exceeds some tolerance in either position or orientation, then the objects are misaligned.

V. Concluding Remarks and Future Work

This paper has presented an in-depth discussion of the current status of NASA's Assemblers project. A mature design for the Stewart platform modules exists along side the assembly test articles which will be used to demonstrate the total system architecture. The multi-agent robotic system is controlled using a task management system, trajectory generation via RRT* with polynomial splines, low-level joint control, an extended Kalman filter sensor fusion system, and a deep learning-based fault detection system. Future work for the project entails integration of all of the subsystems presented here. This integration will culminate in a laboratory ground demonstration of the vertical solar array tower analogue which will be used to validate the mechanical, electrical, and algorithm designs.

The demonstration foundation structure presented here is just one potential approach for a surface assembly worksite dedicated to erecting tower structures with statically positioned manipulators. Other worksite architectures could consider locomotion of the manipulators. This is one avenue for future work beyond the current project. Other possible future efforts include studying the size scalability of Assemblers and further maturation of the technology towards a

flight technology demonstration mission.

Modularity, reconfigurability, and autonomy are key components to the Assemblers architecture and are coupled capabilities that are needed for future human habitation of the lunar surface following the Artemis missions. Assemblers can also play a vital role in construction and maintenance of the lunar gateway and large in-space telescopes planned for future missions.

Acknowledgments

Thanks to NASA team members Benjamin Kelley, Matthew Vaughan, Iok Wong, John Mulvaney, James Ecker, Dr. Walter Waltz, and Emma Brand and project mentors Dr. Danette Allen, Dr. William Doggett, and Eric Cooper.

Also thanks to external collaborators Dr. Erik Komendera of Virginia Tech and Micheal Maksymuk of Honeybee Robotics.

Finally, thanks to the NASA LaRC interns who contributed to the project: Sarah Frey, Sharon Toenjes, Jose Medina, Hamilton Payne, Jacob Martin, Dominic Bisio, Conner Pulling, Jonah Vanke, Nathan Hoege, Jack Pope, and Aaron Shepard.

This work was funded by NASA's Space Technology Mission Directorate as part of the Early Career Initiative program.

References

[1] Hamlin, G., and Sanderson, A., "TETROBOT: a modular approach to parallel robotics," *IEEE Robotics Automation Magazine*, Vol. 4, No. 1, 1997, pp. 42–50. doi:10.1109/100.580984, conference Name: IEEE Robotics Automation Magazine.

[2] Rhodes, M. D., and Mikulas, M. M., Jr., "Deployable Controllable Geometry Truss Beam," Technical Report NASA-TM-86366, L-15901, NAS 1.15:86366, NASA Langley Research Center, Hampton, VA, United States, Jun. 1985. URL https://ntrs.nasa.gov/search.jsp?R=19850019622.

[3] Miura, K., and Furuya, H., "Adaptive Structure Concept for Future Space Applications," *AIAA Journal*, Vol. 26, No. 8, 1988, pp. 995–1002. doi:10.2514/3.10002, URL https://arc.aiaa.org/doi/abs/10.2514/3.10002.

[4] Komendera, E., Dorsey, J. T., Doggett, W. R., and Correll, N., "Truss assembly and welding by Intelligent Precision Jigging Robots," *2014 IEEE International Conference on Technologies for Practical Robot Applications (TePRA)*, 2014, pp. 1–6. doi:10.1109/TePRA.2014.6869150, iSSN: 2325-0526.

[5] Komendera, E. E., and Dorsey, J., "Initial Validation of Robotic Operations for In-Space Assembly of a Large Solar Electric Propulsion Transport Vehicle," *AIAA SPACE and Astronautics Forum and Exposition*, American Institute of Aeronautics and Astronautics, Orlando, FL, 2017. doi:10.2514/6.2017-5248, URL https://arc.aiaa.org/doi/10.2514/6.2017-5248.

[6] Wong, I., Chapin, W., and Komendera, E. E., "A Collaborative Manipulation Strategy for the Assembly of Space Trusses," 2018, p. 6.

[7] Mukherjee, R., Siegler, N., and Thronson, H., "The Future of Space Astronomy will be Built: Results from the In-Space Astronomical Telescope (iSAT) Assembly Design Study," International Astronautical Federation, Washington D.C., United States, 2019, p. 15. URL https://exoplanets.nasa.gov/internal_resources/1375/, iAC-19-F1.2.3.

[8] Belvin, W. K., Doggett, W. R., Watson, J. J., Dorsey, J. T., Warren, J. E., Jones, T. C., Komendera, E. E., Mann, T., and Bowman, L. M., "In-Space Structural Assembly: Applications and Technology," *3rd AIAA Spacecraft Structures Conference*, American Institute of Aeronautics and Astronautics, San Diego, California, USA, 2016. doi:10.2514/6.2016-2163, URL http://arc.aiaa.org/doi/10.2514/6.2016-2163.

[9] Moser, J., Cooper, J., Neilan, J., Chapin, L., Glassner, S., and Komendera, E., "A Reinforcement Learning Approach for the Autonomous Assembly of In-Space Habitats and Infrastructures in Uncertain Environments," International Astronautical Federation (IAF), Washington, D.C., United States, 2019. IAC{19{A5, 3-B3.6, 6, x54219.

[10] Karaman, S., and Frazzoli, E., "Incremental Sampling-based Algorithms for Optimal Motion Planning," *arXiv:1005.0416 [cs]*, 2010. URL http://arxiv.org/abs/1005.0416, arXiv: 1005.0416.

[11] Gilbert, E., Johnson, D., and Keerthi, S., "A fast procedure for computing the distance between complex objects in three-dimensional space," *IEEE Journal on Robotics and Automation*, Vol. 4, No. 2, 1988, pp. 193–203. doi:10.1109/56.2083, conference Name: IEEE Journal on Robotics and Automation.

[12] Lynch, K. M., and Park, F. C., *Modern robotics: mechanics, planning, and control*, Cambridge University Press, Cambridge New York, NY Port Melbourne, 2017. OCLC: 992517931.

[13] Kok, M., Hol, J. D., and Schön, T. B., "Using inertial sensors for position and orientation estimation," *arXiv preprint arXiv:1704.06053*, 2017.

[14] Sabatini, A. M., "Kalman-filter-based orientation determination using inertial/magnetic sensors: Observability analysis and performance evaluation," *Sensors*, Vol. 11, No. 10, 2011, pp. 9182–9206.

[15] MATLAB, *version 1.3.0 (R2020a)*, The MathWorks Inc., Natick, Massachusetts, 2020.

[16] He, K., Gkioxari, G., Dollár, P., and Girshick, R., "Mask r-cnn," *Proceedings of the IEEE international conference on computer vision*, 2017, pp. 2961–2969.

Appendix

A. GJK Algorithm

The main routine and all subroutines for the Gilbert-Johnson-Keerthi algorithm are presented here in pseudocode.

Algorithm 3 Main Gilber-Johnson-Keerthi routine

```
function GJK(A, B)
    Randomly initialize d ∈ ℝ^3
    Initialize simplex, S
    add support(A, B, d) to S
    d ← −d
    while True do
        p ← support(A, B, d)
        add p to S
        if p · d ≤ 0 then
            collision ← False
            break
        else
            d, S ← do_simplex(S)
            if d == None then
                collision ← True
                break
    return collision
```

Algorithm 4 do_simplex subroutine used in the GJK algorithm

```
function DO_SIMPLEX(S)
    A ← most recently added point in S
    B ← second most recently added point in S
    if there are 2 points in S then
        d ← line_case(A, B)
    else if there are 3 points in S then
        C ← third most recently added point in S
        d ← triangle_case(A, B, C)
    else if there are 4 points in S then
        C ← third most recently added point in S
        D ← fourth most recently added point in S
        d ← tetrahedron_case(A, B, C, D)
    return d, S
```

Algorithm 5 line_case subroutine used for updating the simplex

```
function LINE_CASE(A, B)
    AO ← −A
    AB ← B − A
    if AB · AO > 0 then
        d ← (AB × AO) × AB
    else
        d ← AO
    return d
```

Algorithm 6 triangle_case subroutine used to update the simplex

```
function TRIANGLE_CASE(A, B, C)
    AO ← −A
    AB ← B − A
    AC ← C − A
    ABC ← AB × AC
    if (ABC × AC) · AO then
        if AC · AO > 0 then
            d ← (AC × AO) × AC
        else
            d ← line_case(A, B)
    else
        if (AB × ABC) · AO > 0 then
            d ← line_case(A, B)
        else
            if ABC · AO > 0 then
                d ← ABC
            else
                d ← ABC
    return d
```

Algorithm 7 tetrahedron_case used to update the simplex

```
function TETRAHEDRON_CASE(A, B, C, D)
    AO ← −A
    AB ← B − A
    AC ← C − A
    AD ← D − A
    ABC ← AB × AC
    ABD ← AD × AB
    ACD ← AC × AD
    face ← None
    min_dist ← ∞
    for i in [ABC, ABD, ACD] do
        if 0 < i · AO < min_dist then
            face ← i
            min_dist ← i · AO
    if face == ABC then
        d ← triangle_case(A, B, C)
        S ← [C, B, A]
    else if face == ABD then
        d ← triangle_case(A, B, D)
        S ← [D, B, A]
    else if face == ACD then
        d ← triangle_case(A, C, D)
        S ← [D, C, A]
    else
        d ← None
        S ← [D, C, B, A]
    return d, S
```

B. EKF Jacobians

The state matrix for the EKF is given by

$$A_t = \begin{bmatrix} \mathbb{I}_{3\times3} & [0]_{3\times3} & \Delta_t\mathbb{I}_{3\times3} & 0.5\Delta_t^2\mathbb{I}_{3\times3} & [0]_{3\times3} \\ [0]_{4\times3} & \left.\frac{\partial(\Phi q_{B_t}^T)}{\partial q_{B_t}^T}\right|_{\mathbf{x}=\hat{\mathbf{x}}_{t-1|t-1}} & [0]_{4\times3} & 0_{4\times3} & \left.\frac{\partial(\Phi q_{B_t}^T)}{\partial \vec{\omega}_t}\right|_{\mathbf{x}=\hat{\mathbf{x}}_{t-1|t-1}} \\ [0]_{3\times3} & [0]_{4\times3} & \mathbb{I}_{3\times3} & \Delta_t\mathbb{I}_{3\times3} & 0_{3\times3} \\ [0]_{3\times3} & [0]_{4\times3} & 0_{3\times3} & [0]_{3\times3} & \mathbb{I}_{3\times3} \end{bmatrix} \tag{37}$$

where

$$\left.\frac{\partial(\Phi q_{B_t}^T)}{\partial q_{B_t}^T}\right|_{\mathbf{x}=\hat{\mathbf{x}}_{t-1|t-1}} = 0.5\Delta_t \left.\begin{bmatrix} 0 & -\omega_x & -\omega_y & -\omega_z \\ \omega_x & 0 & \omega_z & -\omega_y \\ \omega_y & -\omega_z & 0 & \omega_x \\ \omega_z & \omega_y & -\omega_x & 0 \end{bmatrix}\right|_{\omega=\hat{\omega}_{t-1|t-1}} \tag{38}$$

and

$$\left.\frac{\partial(\Phi q_{B_t}^T)}{\partial \vec{\omega}_t}\right|_{\mathbf{x}=\hat{\mathbf{x}}_{t-1|t-1}} = 0.5\Delta_t \left.\begin{bmatrix} q_1 & -q_2 & -q_3 \\ q_0 & -q_3 & q_2 \\ q_3 & q_0 & -q_1 \\ -q_2 & q_1 & q_0 \end{bmatrix}\right|_{q_{B_t}^T=\hat{q}_{B_{t-1|t-1}}^T} \tag{39}$$

Let $\vec{p}_B = [p_x \;\; p_y \;\; p_z]^\top$, $\vec{m}_T = [m_x \;\; m_y \;\; m_z]^\top$, $\vec{n}_B = [n_x \;\; n_y \;\; n_z]^\top$, and $q_B^T = [q_0 \;\; q_1 \;\; q_2 \;\; q_3]^\top$. The measurement output matrix is given by

$$C_t = \begin{bmatrix} [0]_{3\times3} & [0]_{4\times3} & [0]_{3\times3} & \mathbb{I}_{3\times3} & [0]_{3\times3} \\ [0]_{3\times3} & [0]_{4\times3} & [0]_{3\times3} & [0]_{3\times3} & \mathbb{I}_{3\times3} \\ \left.\frac{\partial\|\vec{l}_p\|}{\partial\vec{p}_B}\right|_{\mathbf{x}=\hat{\mathbf{x}}_{t-1|t-1}} & \left.\frac{\partial\|\vec{l}_p\|}{\partial q_B^T}\right|_{\mathbf{x}=\hat{\mathbf{x}}_{t-1|t-1}} & [0]_{1\times3} & [0]_{1\times3} & [0]_{1\times3} \end{bmatrix} \tag{40}$$

where

$$\left.\frac{\partial\|\vec{l}_p\|}{\partial\vec{p}_B}\right|_{\mathbf{x}=\hat{\mathbf{x}}_{t-1|t-1}} = g(\hat{\vec{p}}_{B_{t-1|t-1}}, \hat{q}_{B_{t-1|t-1}}^T)^{-1/2} \left.\begin{bmatrix} (p_x+m_x-n_x) \\ (p_y+m_y-n_y) \\ (p_z+m_z-n_z) \end{bmatrix}\right|_{\vec{p}_B=\hat{\vec{p}}_{B_{t-1|t-1}}} \tag{41}$$

and

$$\left.\frac{\partial\|\vec{l}_p\|}{\partial q_B^T}\right|_{\mathbf{x}=\hat{\mathbf{x}}_{t-1|t-1}} = g(\hat{\vec{p}}_{B_{t-1|t-1}}, \hat{q}_{B_{t-1|t-1}}^T)^{-1/2} * \tag{42}$$

$$\left.\begin{bmatrix} (p_x+m_x-n_x)(-2m_yq_3+2m_zq_2)+(p_y+m_y-n_y)(2m_xq_3-2m_zq_1) \\ +(p_z+m_z-n_z)(-2m_xq_2+2m_yq_1) \\ (p_x+m_x-n_x)(2m_yq_2+2m_zq_3)+(p_y+m_y-n_y)(2m_xq_2+2m_zq_3) \\ +(p_z+m_z-n_z)(2m_xq_3+2m_yq_0-4m_zq_2) \\ (p_x+m_x-n_x)(-4m_xq_2+2m_yq_1+2m_zq_0)+(p_y+m_y-n_y)(2m_xq_1+2m_zq_3) \\ +(p_z+m_z-n_z)(-2m_xq_0+2m_yq_3-4m_zq_2) \\ (p_x+m_x-n_x)(-4m_xq_3-2m_yq_0+2m_zq_1)+(p_y+m_y-n_y)(2m_xq_0-4m_yq_3+2m_zq_2) \\ +(p_z+m_y-n_z)(2m_xq_1+2m_zq_2) \end{bmatrix}\right|_{q_{B_t}^T=\hat{q}_{B_{t-1|t-1}}^T} \tag{43}$$

where $g(\hat{\vec{p}}_{Bt-1|t-1}, \hat{q}_{B_{t-1|t-1}}^T) = \|\hat{\vec{p}}_{B_{t-1|t-1}} + q_{B_{t-1|t-1}}^{T*} \odot \vec{m} \odot q_{B_{t-1|t-1}}^T - \vec{n}\|^2 + \epsilon$.